HOPSWORKS

ID2223 · KTH · GUEST LECTURE · 12 NOVEMBER 2026

Modular AI Systems with Hopsworks

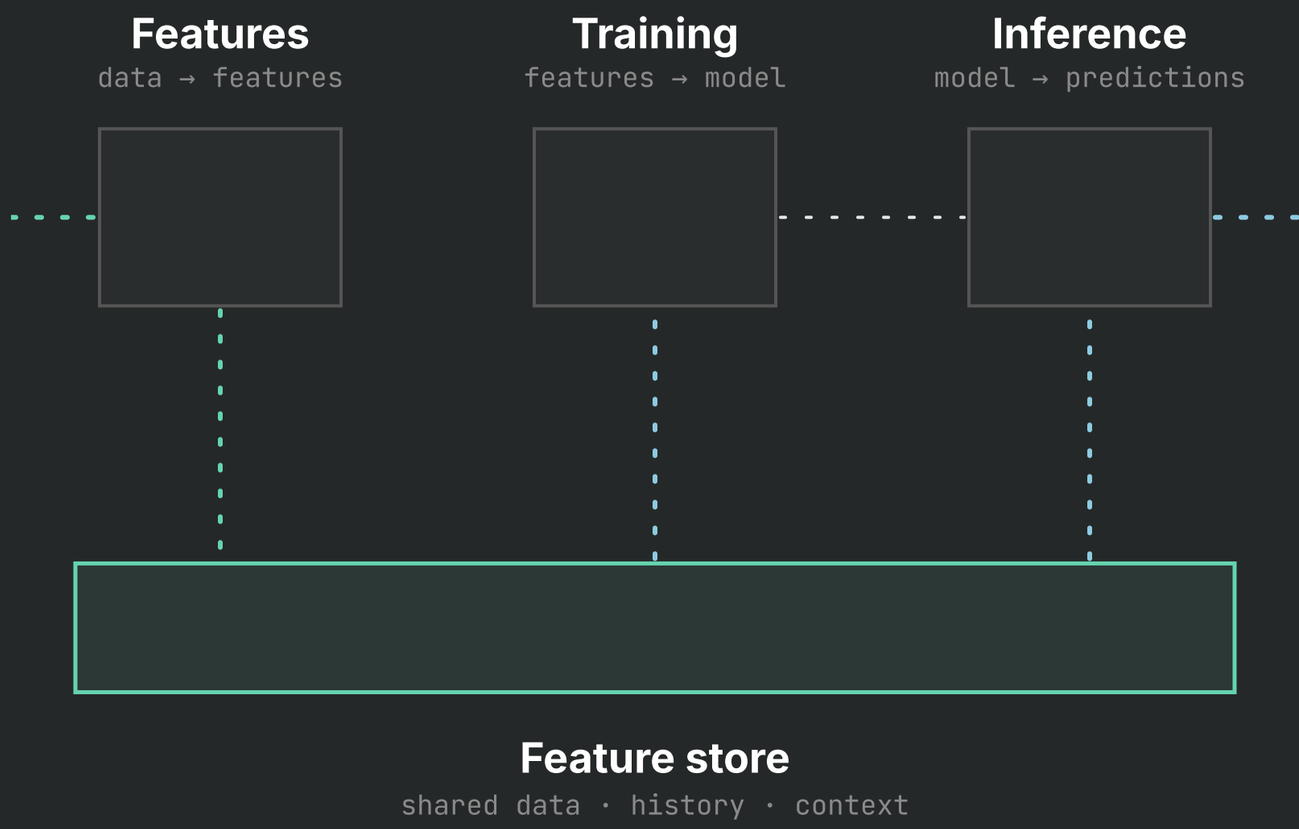
A unified architecture for batch, real-time and LLM AI systems with a feature store



Jim Dowling, PhD

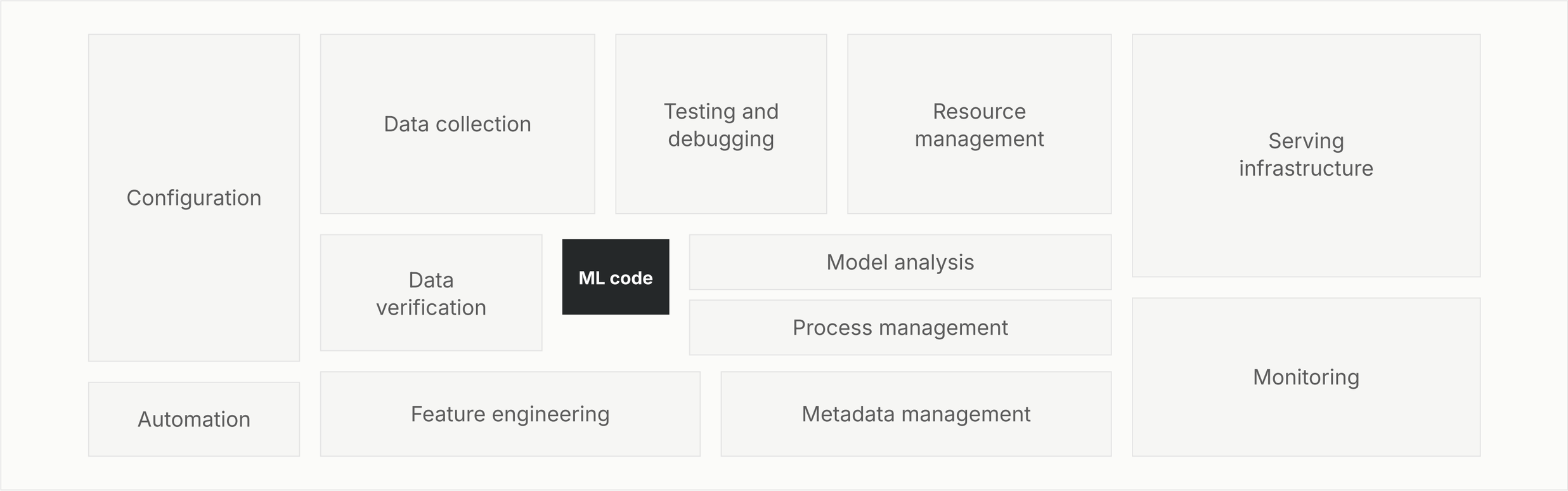
CEO & CO-FOUNDER, HOPSWORKS

slides.hops.io/2026-11/id2223 COPY



HW

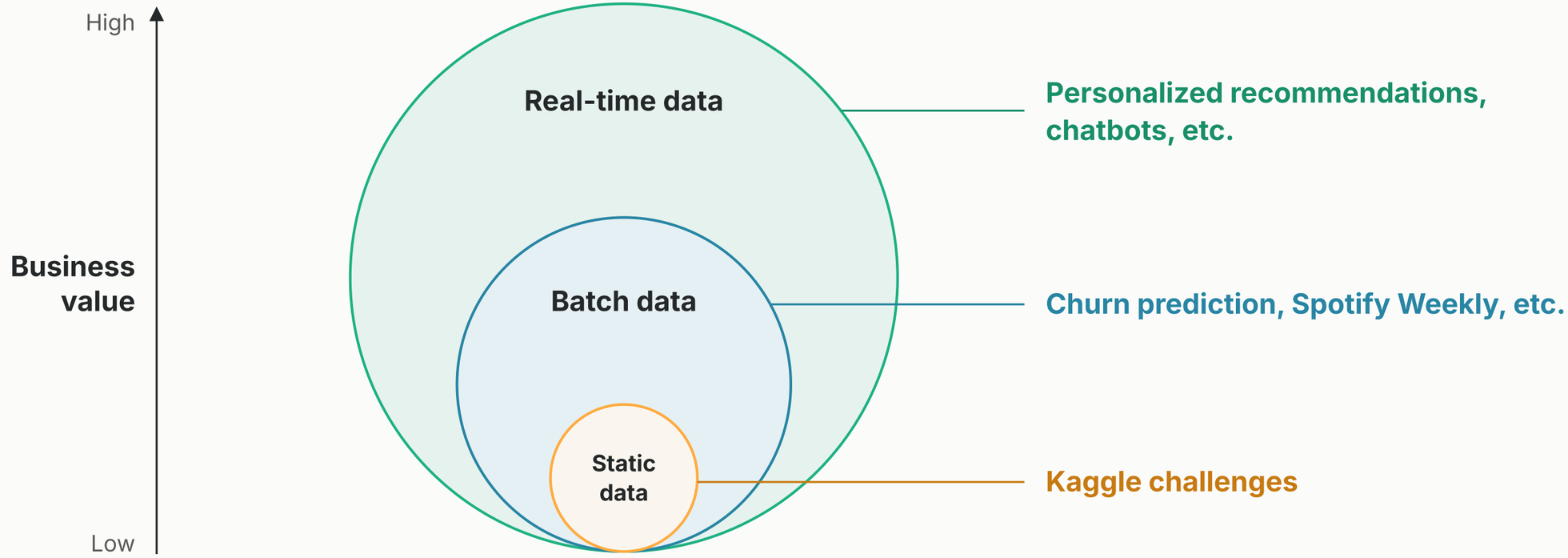
// Pre-MLOps: move fast and ship it



Only a small fraction of a real-world ML system is ML code. The rest is the system around it, and its technical debt.

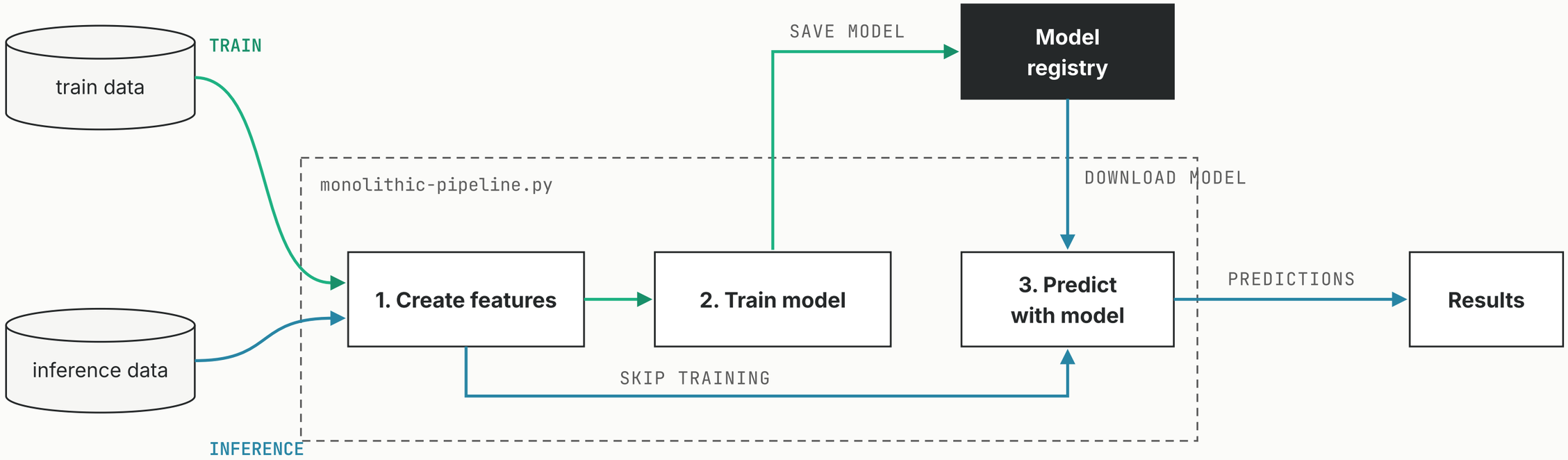
Figure after Sculley et al., Hidden Technical Debt in Machine Learning Systems, NeurIPS 2015, as drawn in Google Cloud's MLOps: continuous delivery and automation pipelines

// AI systems typically work on dynamic data



Real-time data: the highest business value, predictions as the data arrives.

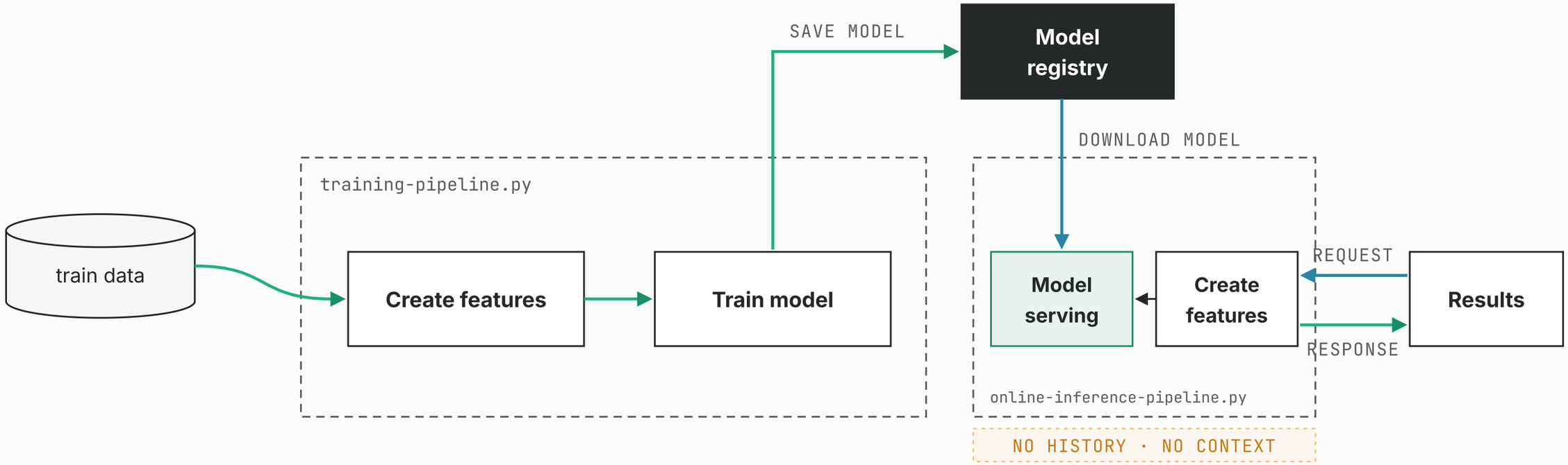
// Monolithic AI pipelines kind of work for batch AI systems



...or in **INFERENCE** mode, with the model downloaded from the registry.

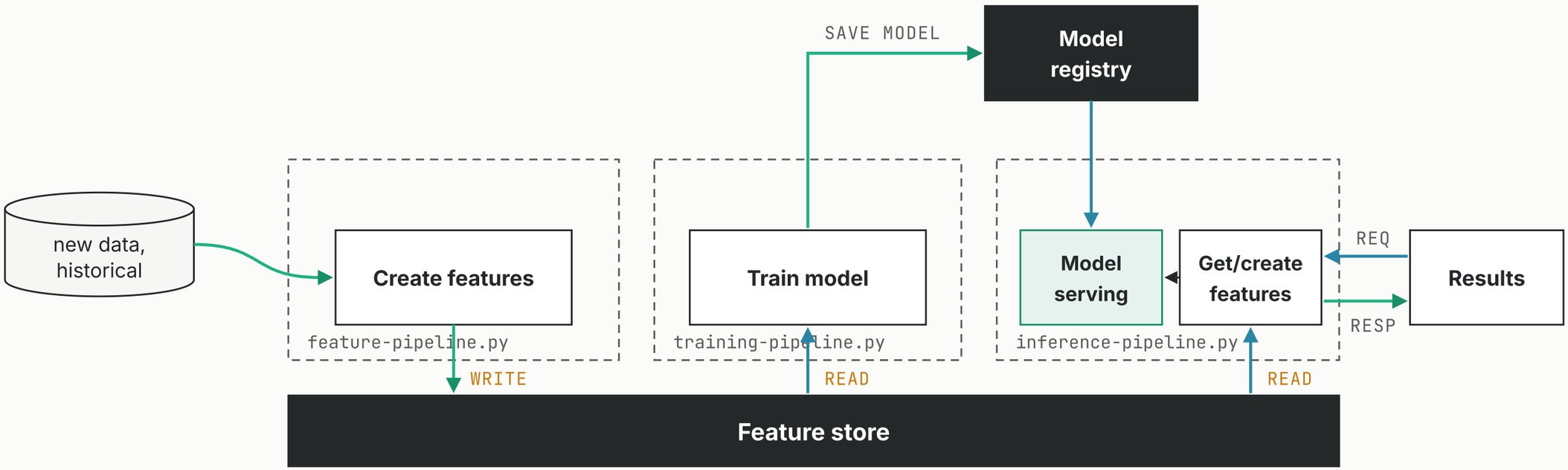
HW

// There is no monolithic AI pipeline for real-time AI systems



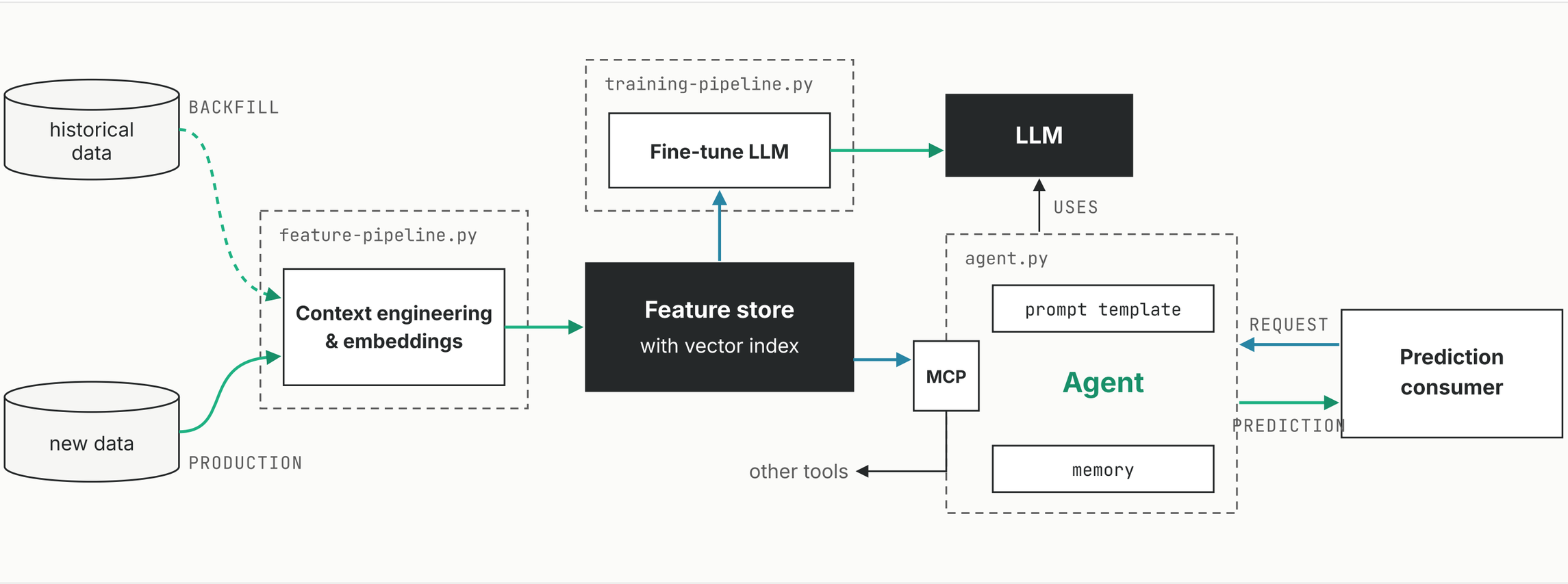
Online models are stateless. To give them history and context, add a **feature store**.

// Now there are three AI pipelines, with a feature store

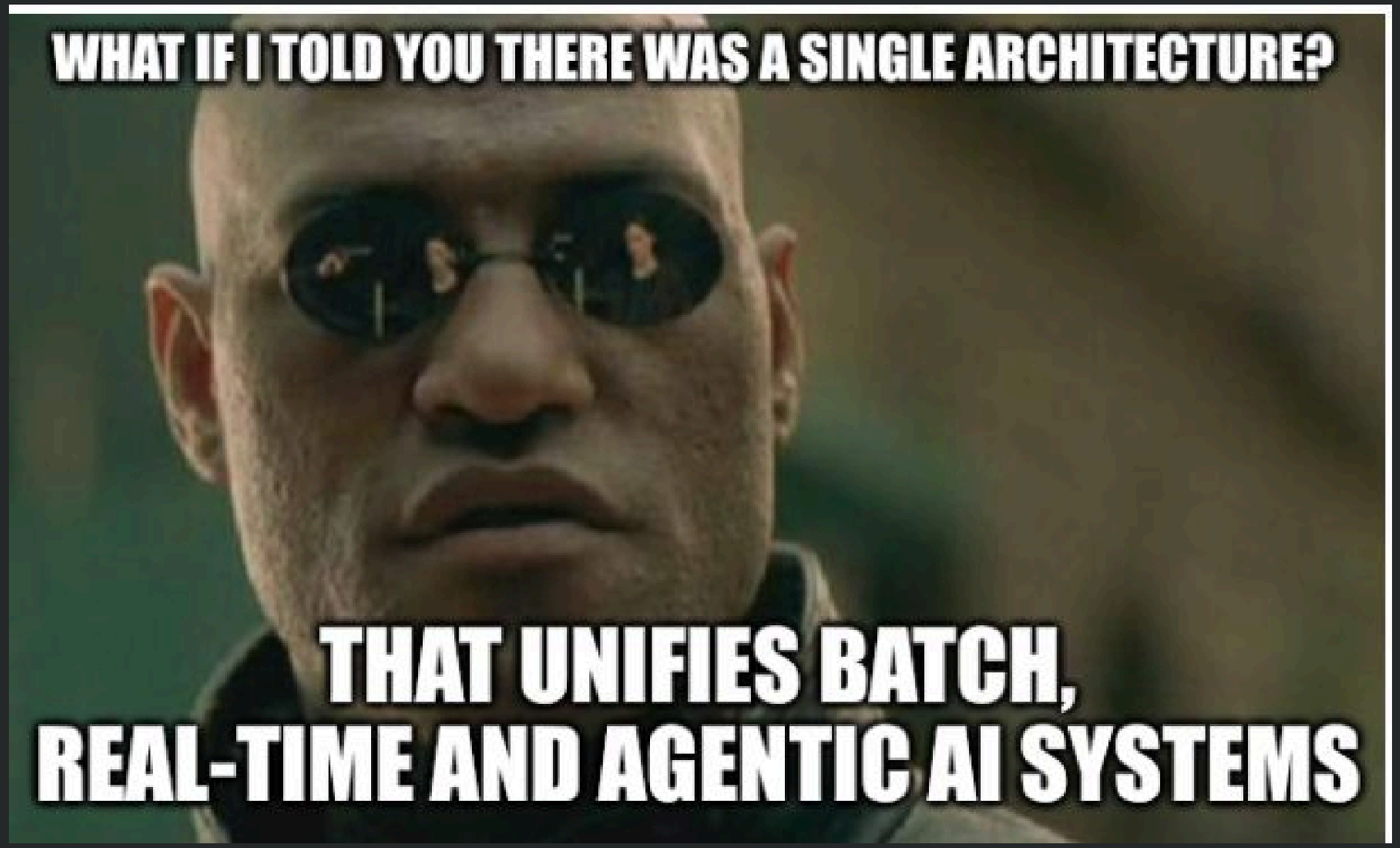


3. An online **model serving** service gets precomputed features, creates request-time ones, and responds.

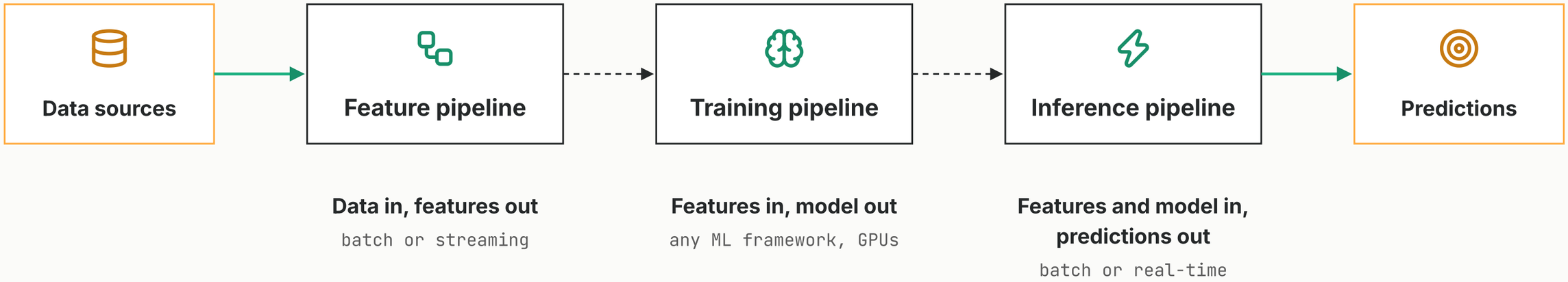
// Agentic systems also require pipelines



The **agent** is the inference pipeline: a prompt template, memory, and tools over MCP, the feature store among them.

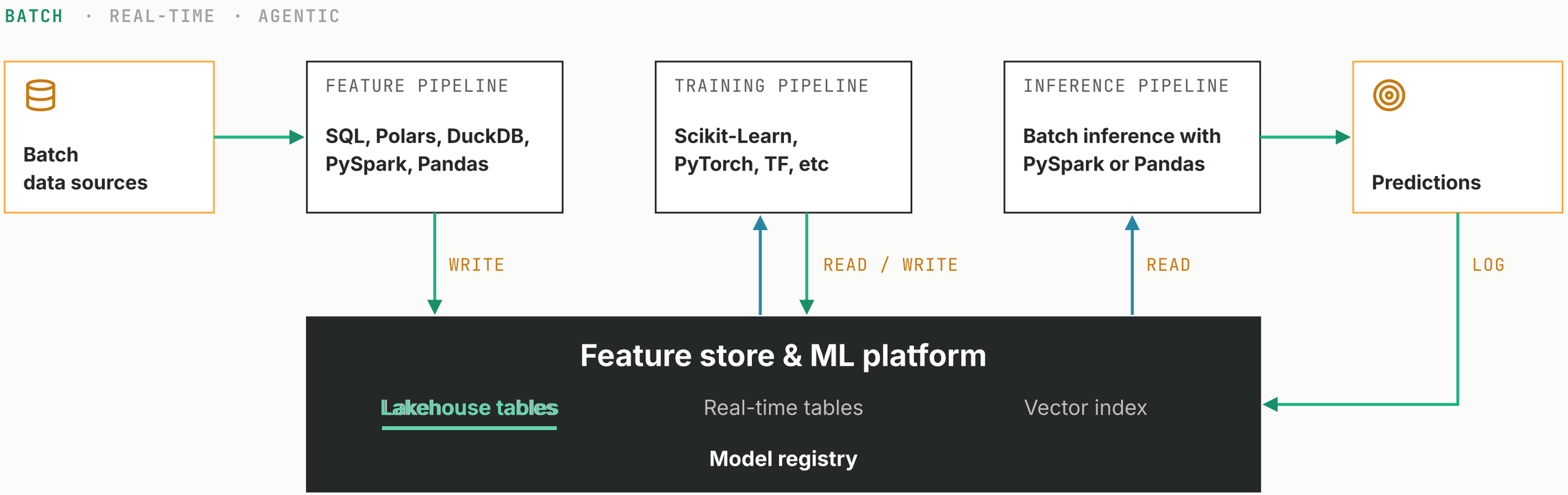

WHAT IF I TOLD YOU THERE WAS A SINGLE ARCHITECTURE?
THAT UNIFIES BATCH,
REAL-TIME AND AGENTIC AI SYSTEMS

// A unified architecture for batch, real-time and LLM AI systems



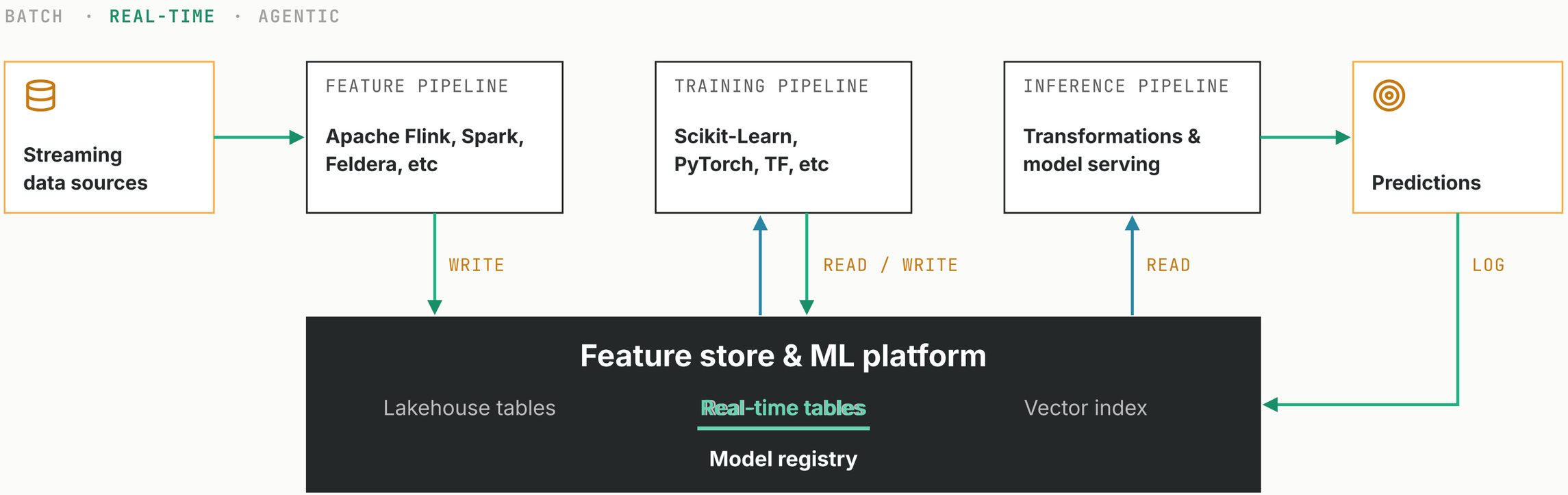
The **inference pipeline** turns features and a model into predictions.

// Batch AI Systems



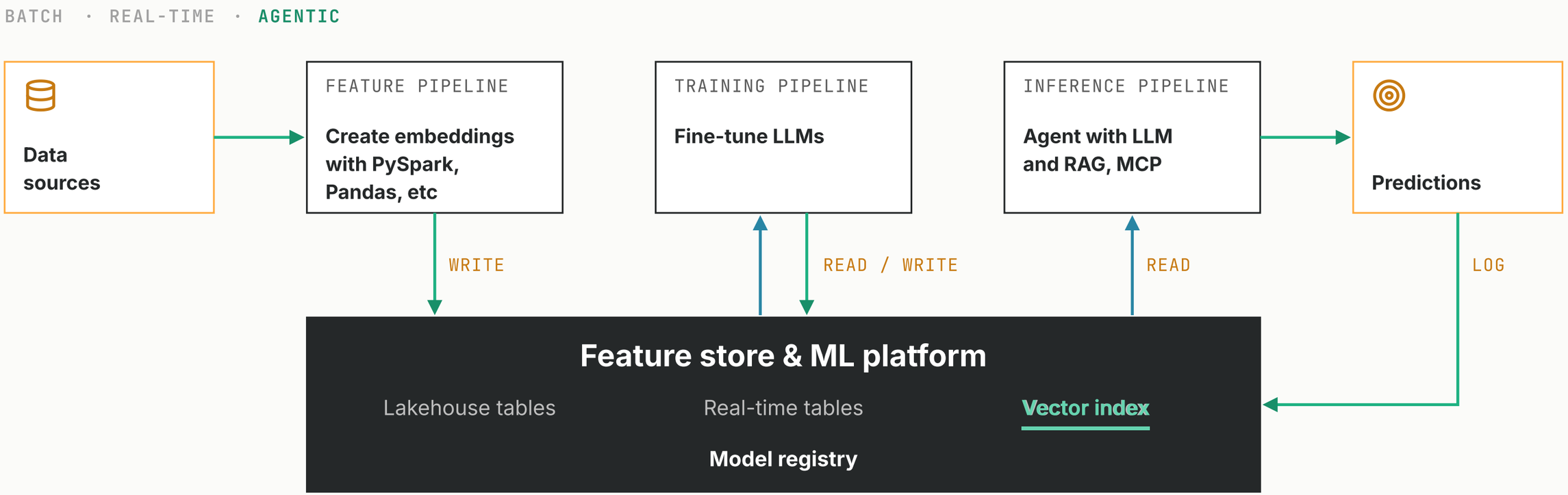
Batch AI systems: batch sources, features in lakehouse tables, batch inference.

// Real-Time AI Systems



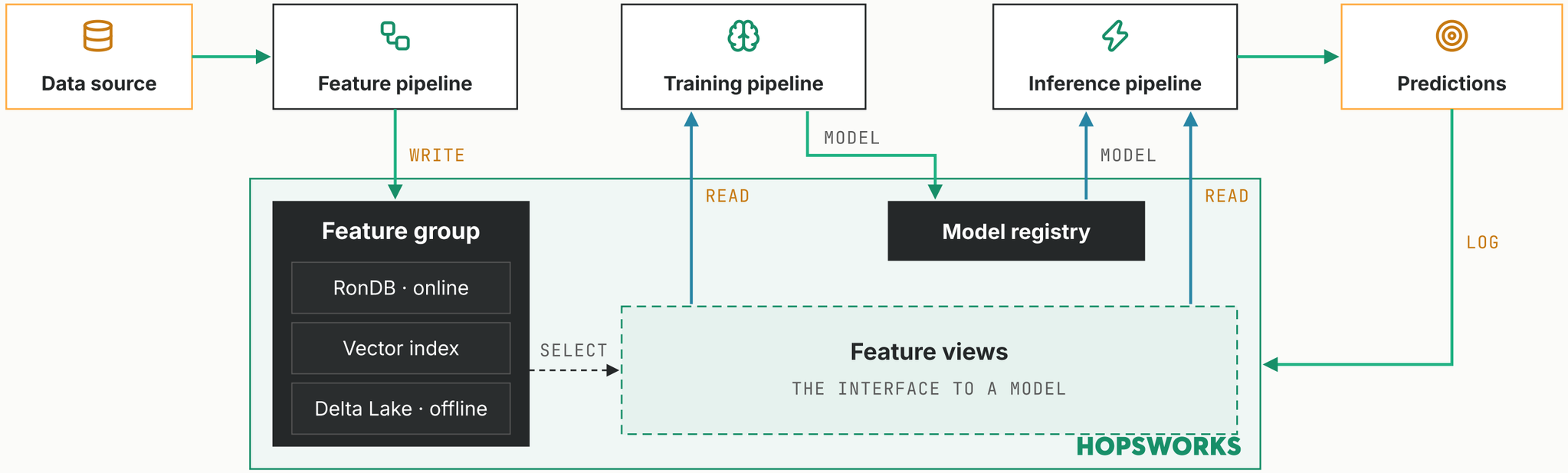
Real-time AI systems: streaming sources, fresh features in real-time tables, online model serving.

// Agentic AI Systems



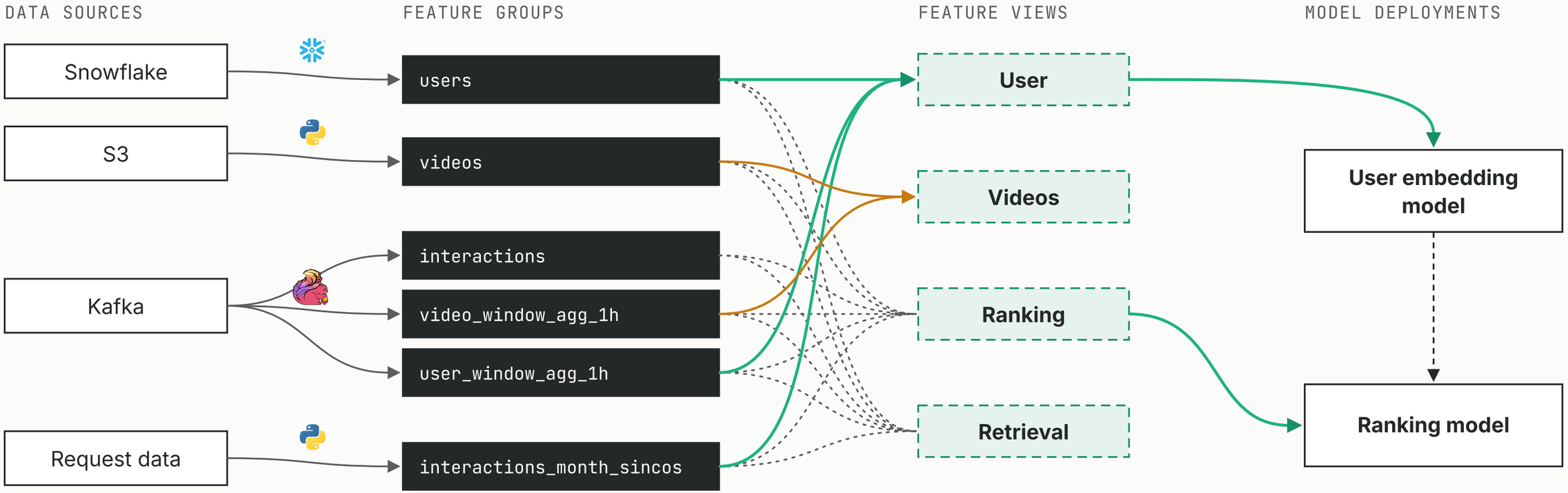
Agentic systems: embeddings in a vector index, fine-tuned LLMs, an agent with RAG and MCP.

// Hopsworks: write to feature groups, read from feature views



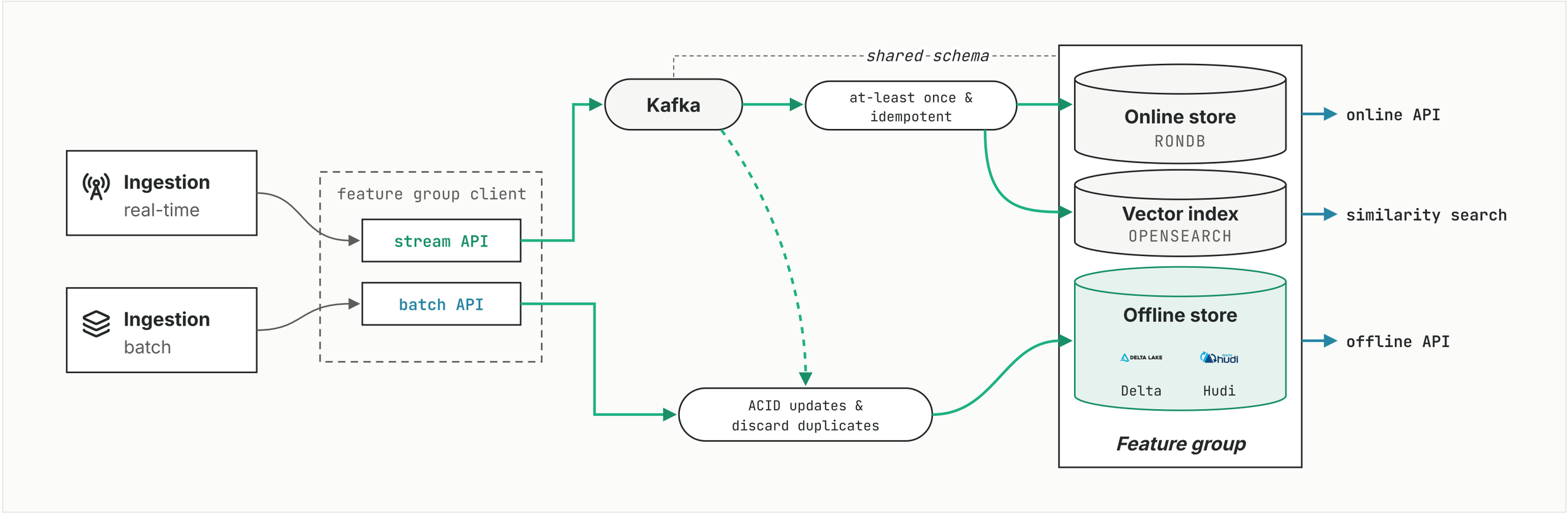
Inference pipelines read the same feature view and the model. Predictions are logged.

// From features to models



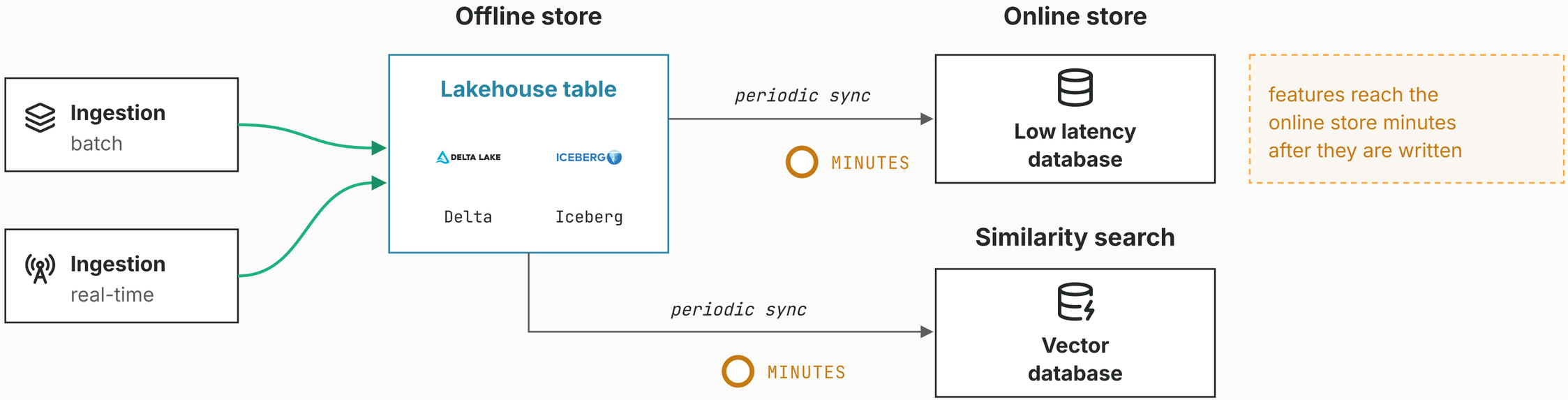
Each feature view is the interface to a model: here a user embedding model and a ranking model.

// Hopsworks: a feature store for real-time AI systems



One feature group, three APIs. You can build real-time AI systems on this.

// Other feature store architectures: Databricks, Vertex



Poor feature freshness: a periodic sync takes minutes to bring features to the online store.

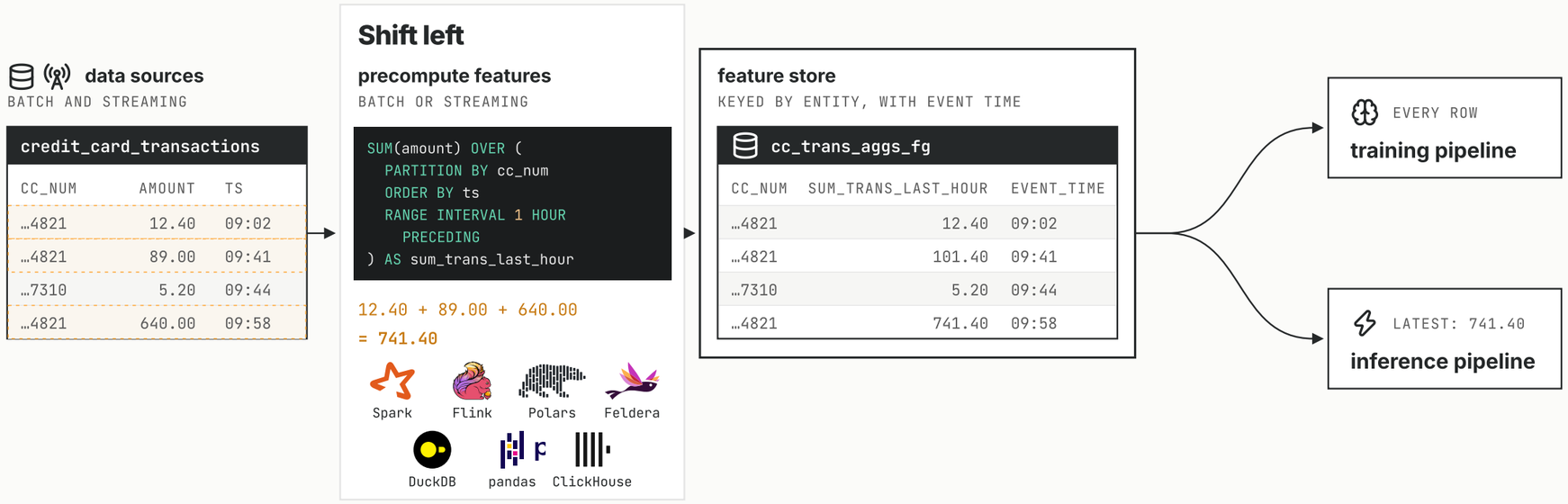
HW

01

Feature pipelines

FEATURE → TRAINING → INFERENCE

Feature pipelines transform and store feature data in a feature store



Training data is a batch of historical feature data, inference the latest. The **same precomputed values**, no skew.

HW

// Write to feature groups from a Pandas, Polars or PySpark DataFrame

```python
import hopsworks

project = hopsworks.login()
fs = project.get_feature_store()

df = # Feature engineering to create a fraud transactions DataFrame

trans_fg = fs.get_or_create_feature_group(
    name="transactions_fraud_online_fg",
    version=1,
    description="Transaction data",
    primary_key=['cc_num'],
    event_time='datetime',
    online_enabled=True,
    expectation_suite=expectation_suite_transactions,
)
trans_fg.insert(df)
```

`insert(df)` : one call writes the DataFrame, offline and online.

HW

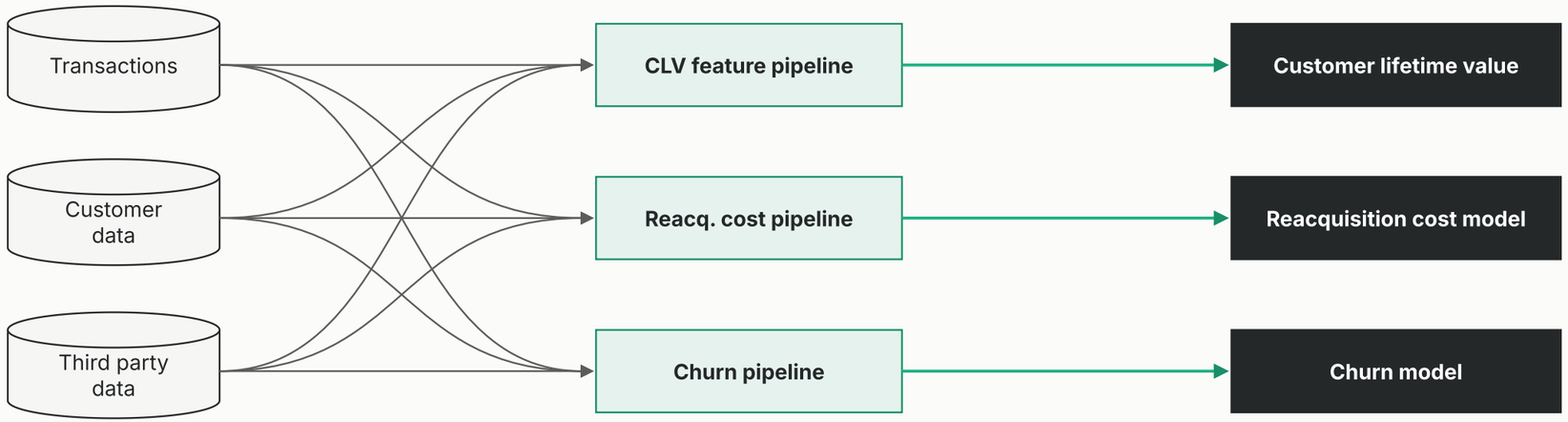
// With no feature store, each model needs its own feature pipeline

PROBLEM

Different models re-compute the same features. This leads to waste, and a less maintainable (non-DRY) code base.

FEATURE STORE SOLUTION

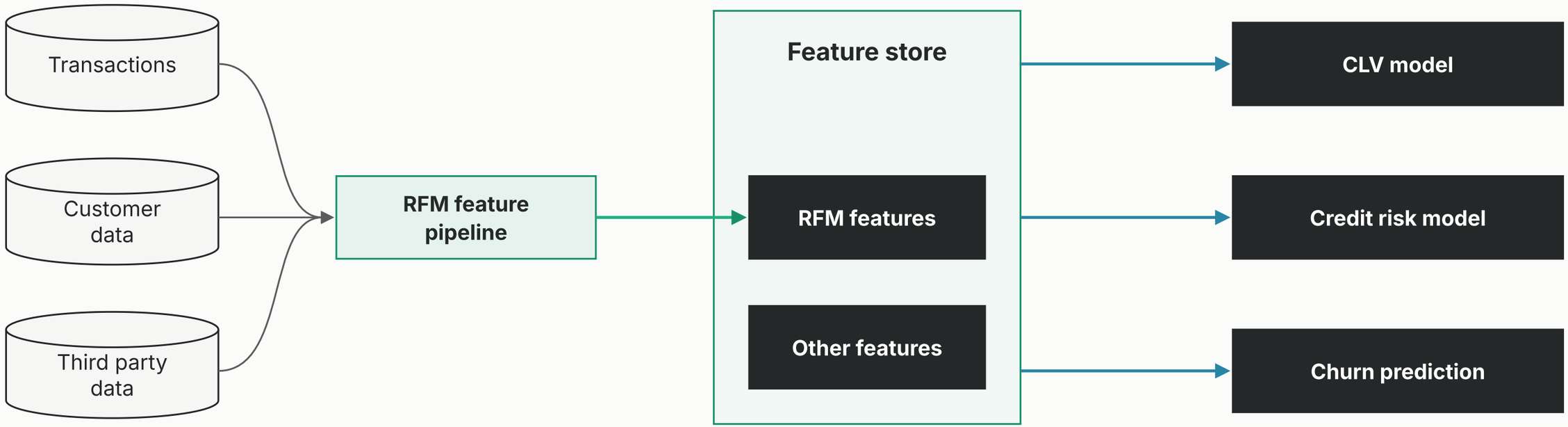
Store precomputed features in feature groups (tables) that models reuse.



...and the third. A new model means a new feature pipeline, for training and for serving. No reuse.

HW

// Reuse features across models by storing them in feature tables



...and the models after it reuse them. A new model selects existing features: no new feature pipeline to write and test.

HW

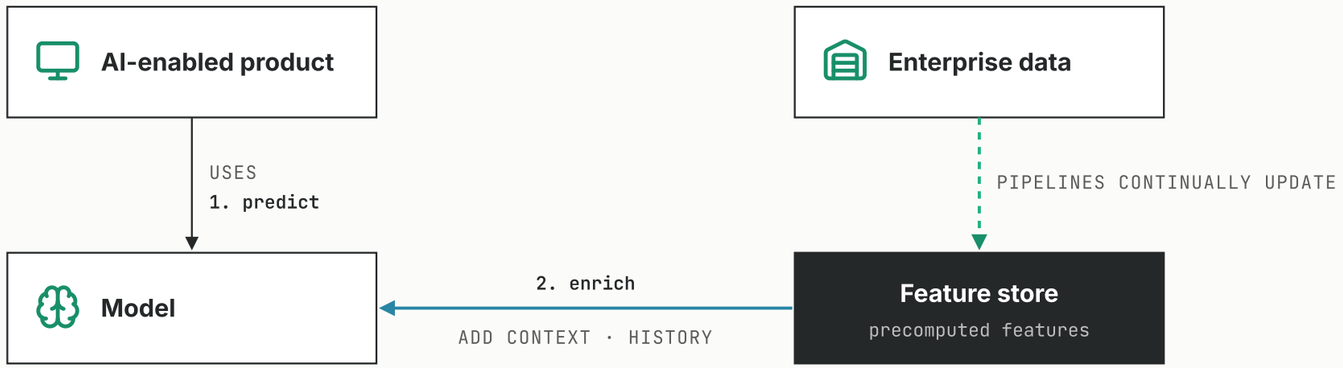
// Feature stores provide history and context to online models

PROBLEM

Your AI-enabled application uses an online model that is stateless. It needs historical and context data to make better predictions.

FEATURE STORE SOLUTION

The online feature store serves precomputed features that give the model history and context, at low latency.



2. The feature store enriches the request with the history and context the model needs.

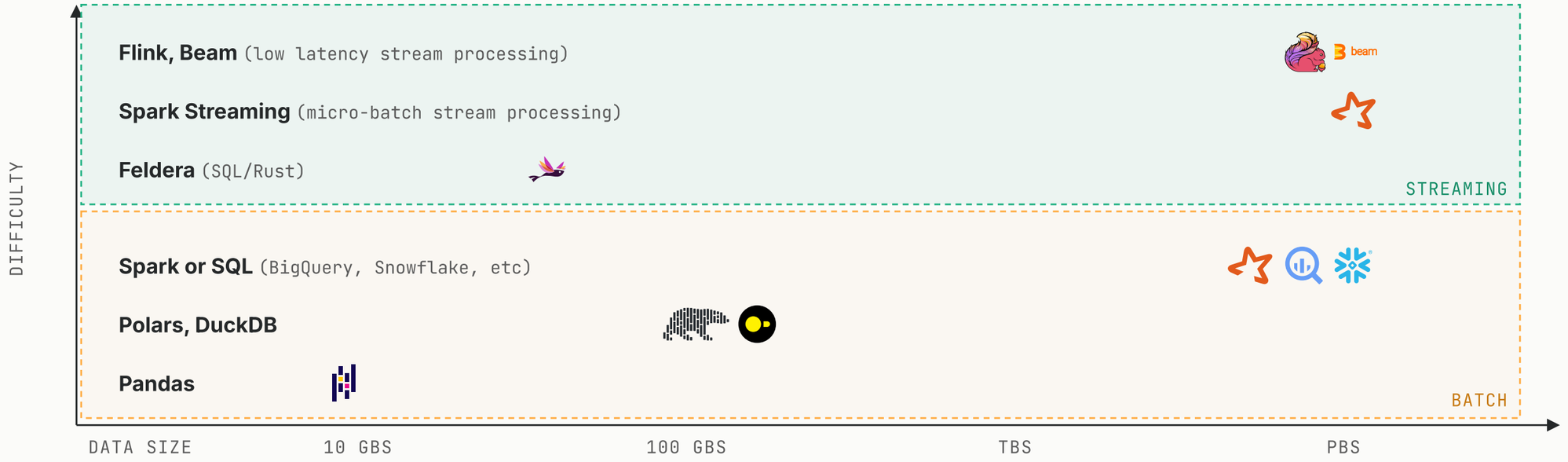
// Support the best data processing framework for creating features

PROBLEM

The best query engine depends on your data volumes, feature freshness requirements, and rolling aggregation requirements.

FEATURE STORE SOLUTION

Provide an open architecture for feature pipelines, both batch and streaming.



HW

02

Training pipelines

FEATURE → TRAINING → INFERENCE

HW

// Feature views and training data

```python
# Create feature view for model
trans_fg = fs.get_feature_group(name='transactions_fraud_online_fg', version=1)
profile_online_fg = fs.get_feature_group(name='profile_fraud_online_fg', version=1)

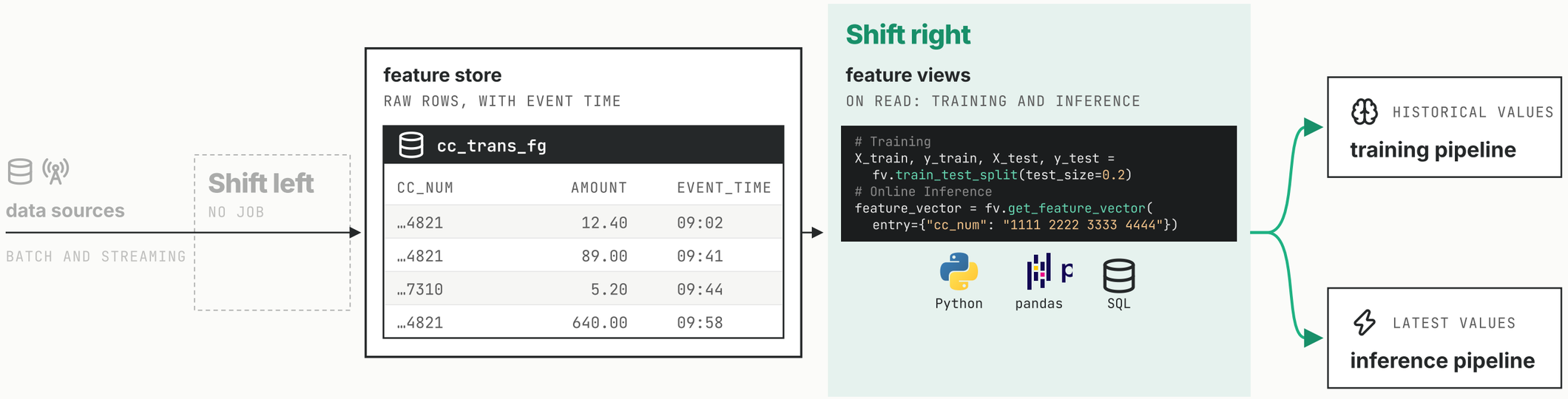
selected_features = trans_fg.select_features().join(profile_online_fg.select_all())

fv = fs.get_or_create_feature_view(
    name='transactions_fraud_online',
    version=1,
    query=selected_features,
    labels=["fraud_label"],
    transformation_functions=[log_amount("amount").alias("amount_log")],
    logging_enabled=True,
)

# Read training data
X_train, X_test, y_train, y_test = fv.train_test_split(test_size=0.2)
```

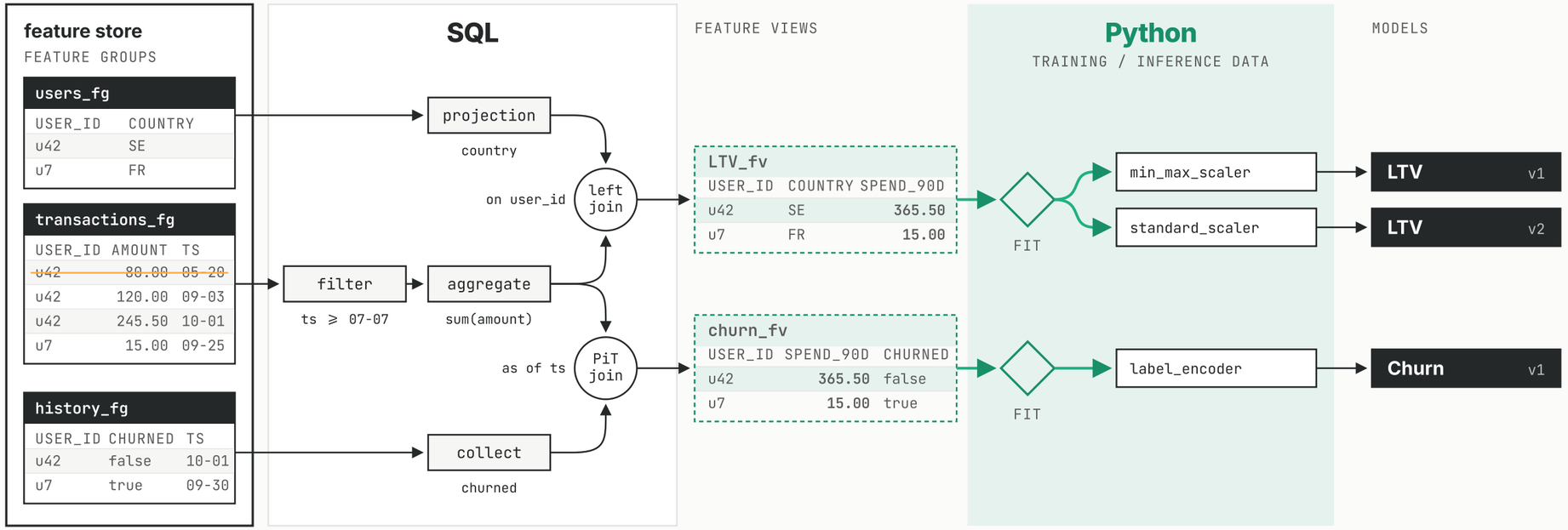
`train_test_split`: point-in-time correct training data, split into train and test sets.

The feature view transforms on read



Shift right: the feature view retrieves data and transforms read data.

Feature Views read with SQL, then Transform in Python



A model is trained on one version and served with **the same transformations**.

TRANSFORMATION DAG · A DATAFLOW GRAPH, AS IN APACHE HAMILTON

The DAG is in the parameter names

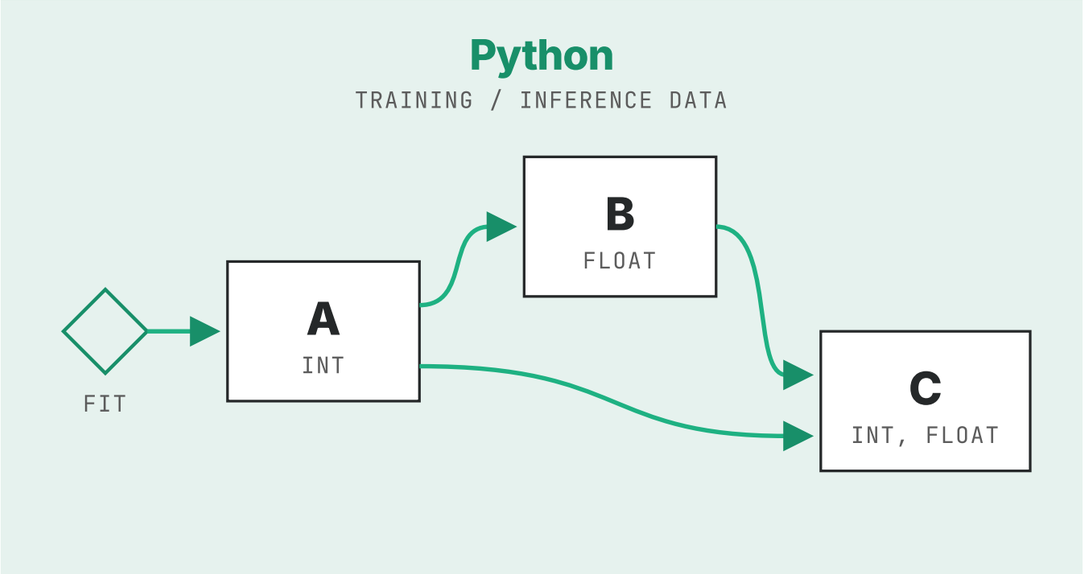
transformations.py

```python
@hopsworks.udf(return_type=[int])
def A() -> int:
    """Constant value 35"""
    return 35

@hopsworks.udf(return_type=[float])
def B(A: int) -> float:
    """Divide A by 3"""
    return A / 3

@hopsworks.udf(return_type=[int, float])
def C(A: int, B: float) -> tuple[int, float]:
    """Ceiling of B, and A squared times B"""
    return math.ceil(B), A**2 * B

fv = fs.create_feature_view(...,
    transformation_functions=[A(), B("A"), C("A", "B")],
)
```



The feature view takes the DAG as a list: each function once, its inputs named. The order of evaluation follows from the names.

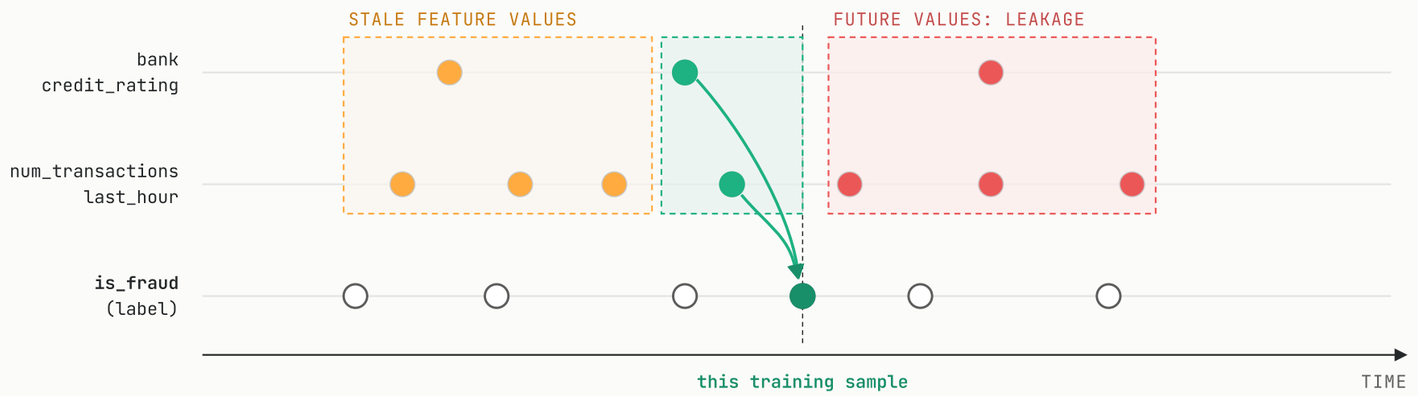
// Prevent data leakage with temporal joins across feature tables

PROBLEM

It is hard to create training data and batch inference data from tabular time-series data without data leakage or stale feature data.

FEATURE STORE SOLUTION

Feature stores create point-in-time consistent snapshots of mutable feature data.



Older values are stale. Any later value is future data: include it and you leak the future into training.

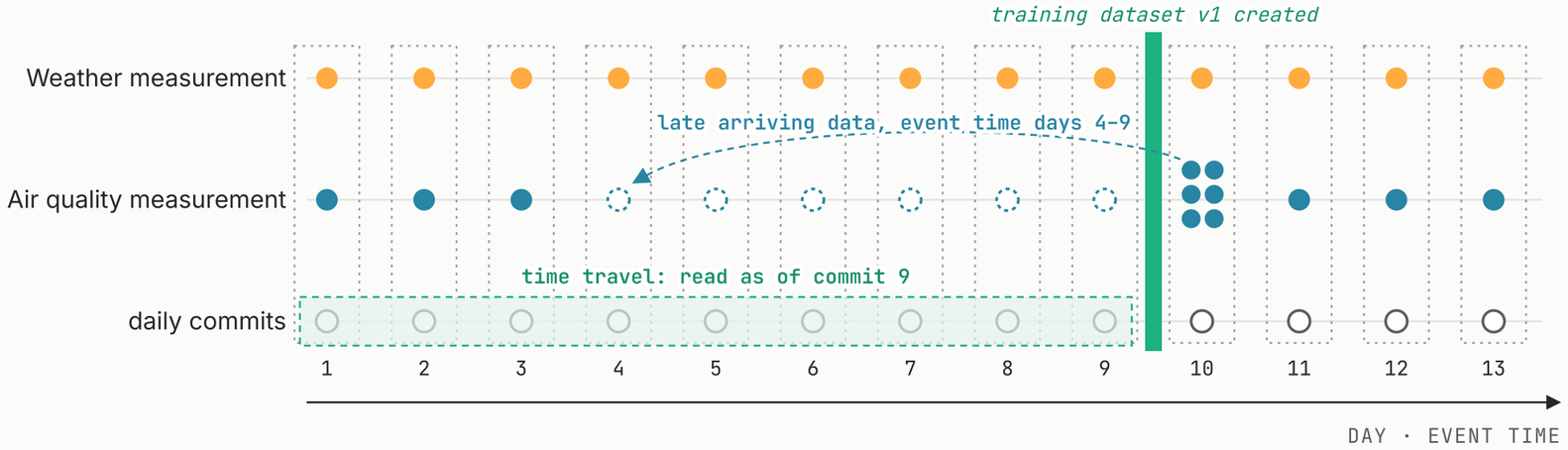
// Reproducible training data with time travel in lakehouse tables

PROBLEM

Re-create training data using only **event time**, and **late arriving data** means you will not re-create it correctly.

FEATURE STORE SOLUTION

Use **ingestion time**. Lakehouse tables keep it for every commit, and **time travel** re-reads old commits.



Time travel re-reads the table as of commit 9: exactly the rows v1 was created from.

HW

03

Inference pipelines

FEATURE → TRAINING → INFERENCE

HW

// Feature views and inference

```
fv = fs.get_or_create_feature_view(
    name='transactions_fraud_online',
    version=1)

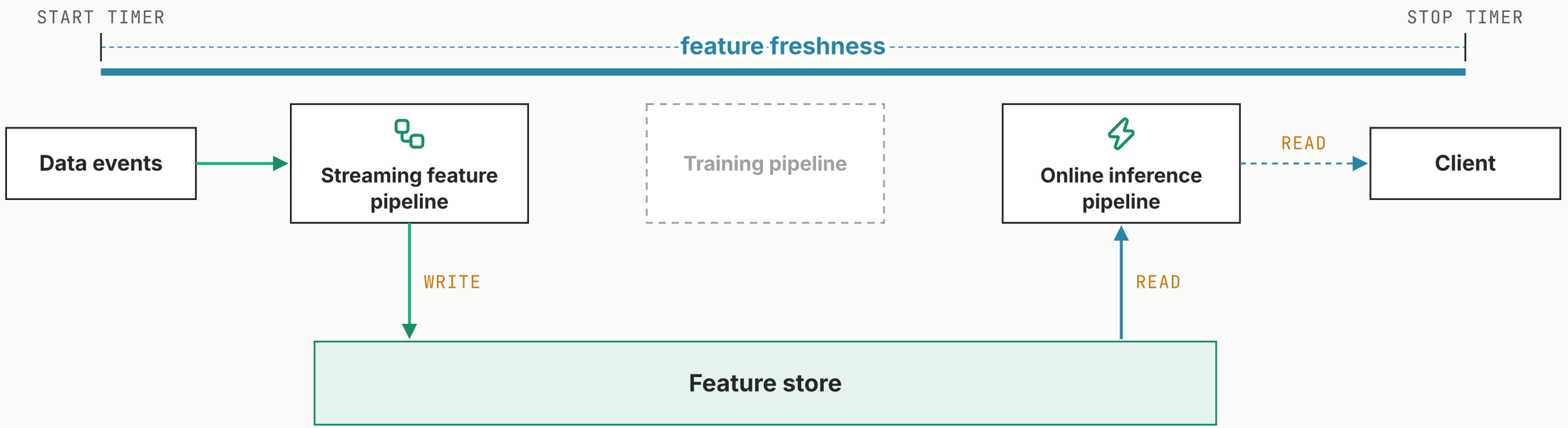
# Batch inference
df = fv.get_batch_data(start_time=yesterday)
pred_df = model.predict(df)

# Online inference
feature_vector = fv.get_feature_vector(serving_keys={"cc_num": 1234})
pred = model.predict(feature_vector)
```

Online inference:
`get_feature_vector` reads one row from the online store, by its serving key.

HW

// What is feature freshness?



TikTok has feature freshness of 1-2 secs. It is their main competitive advantage.

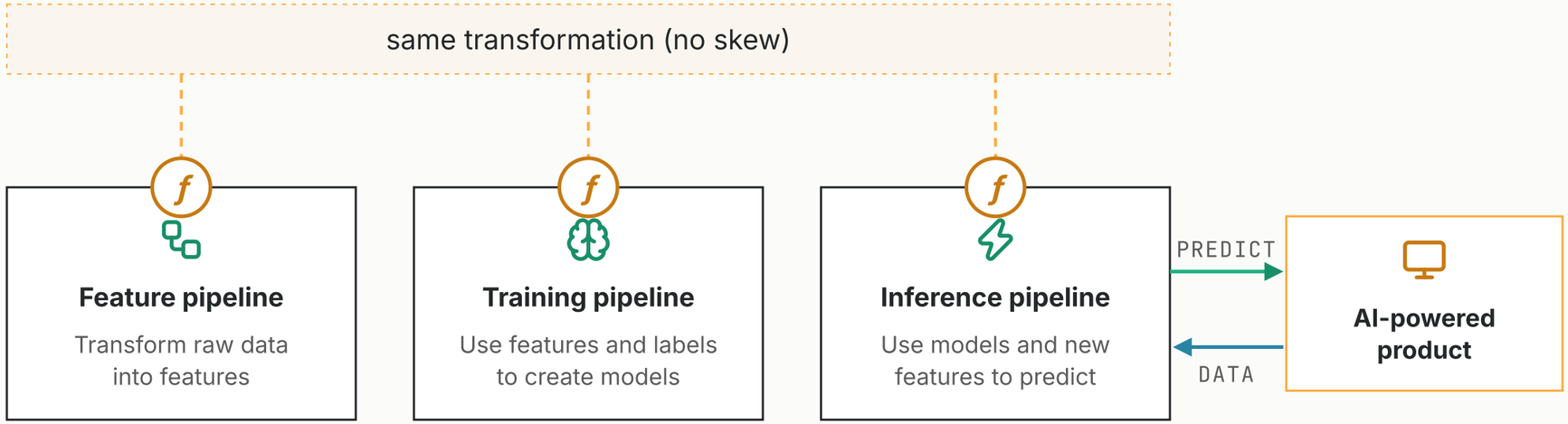
// Feature functions

PROBLEM

You want to use a transformation in your inference pipeline, and reuse the same transformation in your feature or training pipeline.

FEATURE STORE SOLUTION

Implement transformations as feature functions, reused in the inference pipeline and in the training or feature pipeline.



Write the transformation once, as a feature function, and use it in every pipeline: no skew.

// Transformation functions in Hopsworks

features/air_quality.py

```
@hopsworks.udf(return_type=int)
def moving_average(df, window=7):
    df[f'mean_{window}_days'] = df.groupby('city_name')['pm2_5'] \
          .rolling(window=window).mean().reset_index(0,drop=True).shift(1)
```

The moving average of PM2.5 per city, shifted by one day so that it uses only past values.

// Feature monitoring and alerts

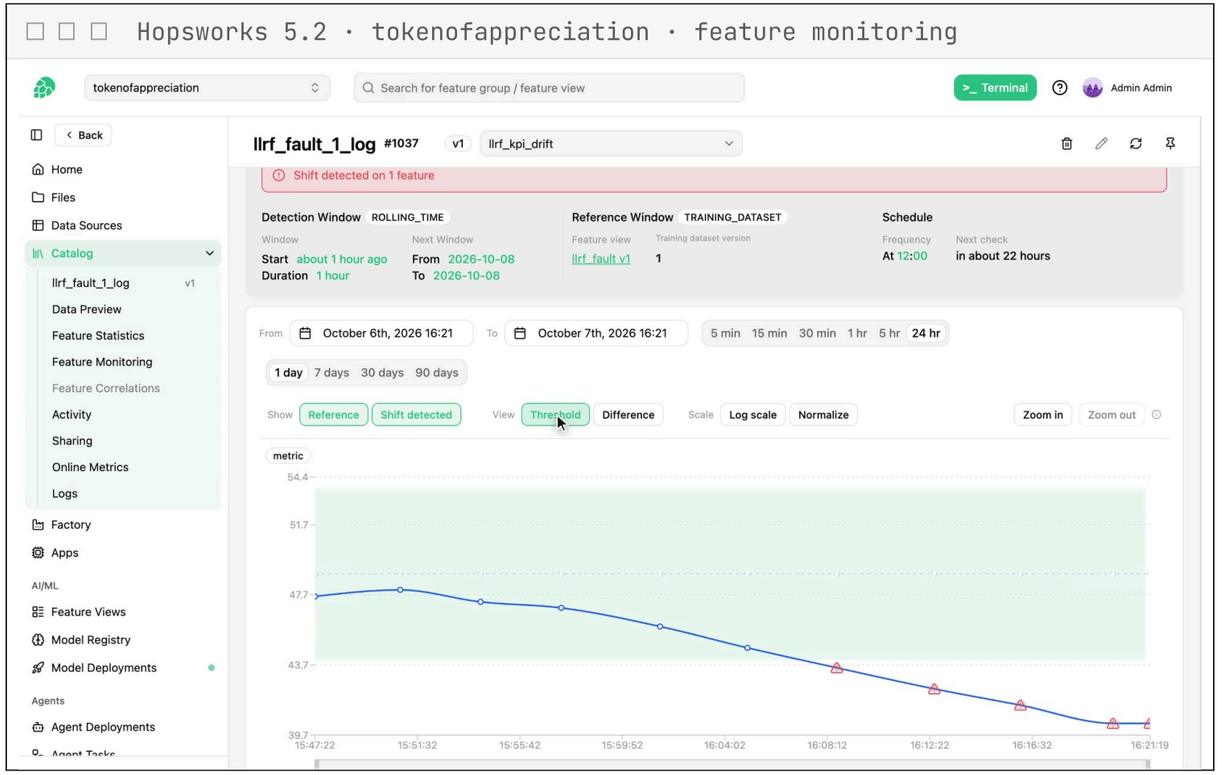
PROBLEM

Model performance in production can degrade unexpectedly.

FEATURE STORE SOLUTION

Automate feature and model monitoring with feature and prediction logging. Log both encoded and unencoded feature values.

```
user_search_queries_fg.enable_feature_monitoring(
    name="brand_name_monitoring",
    feature_name="has_brand_name",
    job_frequency="DAILY",
).with_detection_window(
    time_offset="1d",
    row_percentage=0.1,
).with_reference_window(
    specific_value=td.statistics.feature.mean
).compare_on(
    metric="mean",
    relative=True,
    threshold=0.5,
).save()
```



HW

04

Agents

FEATURE → TRAINING → INFERENCE

// Write vector embeddings to a feature group

```
model = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")
documents = [{"doc_id": 1, "text": "Suspicious credit card transaction"}]

df = pd.DataFrame(documents)
df["embedding"] = model.encode(df["text"].tolist()).astype("float32").tolist()

emb_index = embedding.EmbeddingIndex()
emb_index.add_embedding(name="embedding", dimension=384)

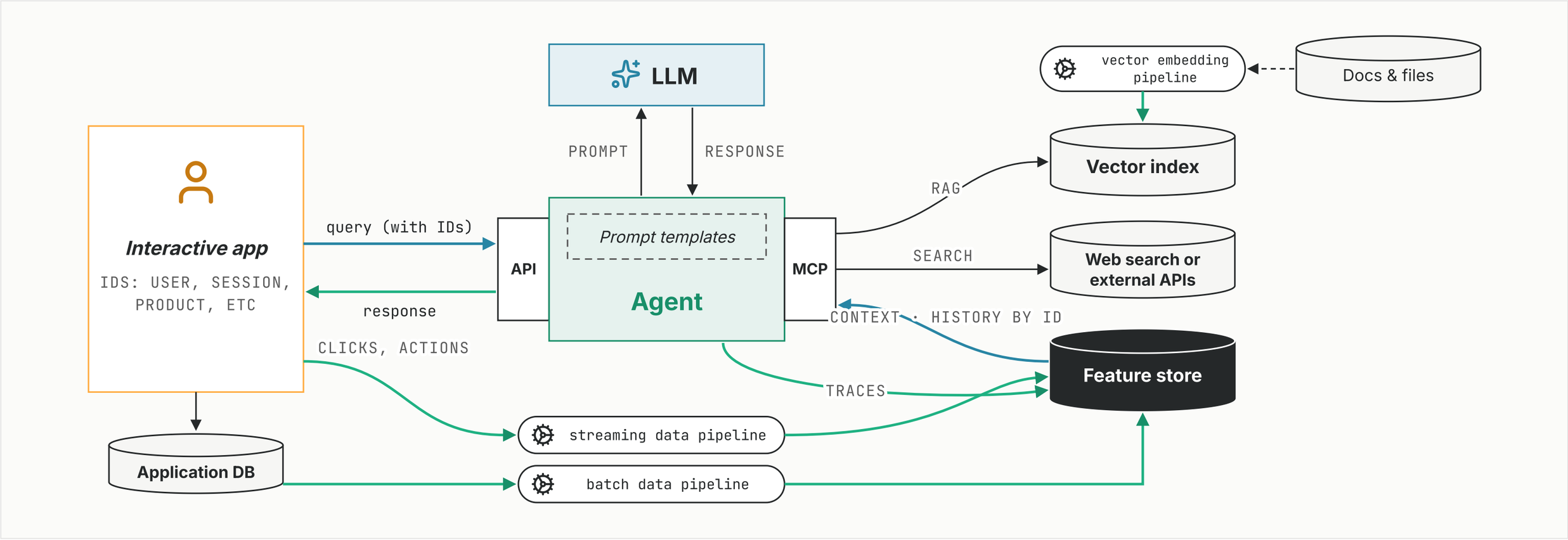
fg = fs.get_or_create_feature_group(name="document_embeddings",
    embedding_index=emb_index,
    primary_key=["doc_id"],
    online_enabled=True
)
fg.insert(df)

# Vector similarity search
query_embedding = model.encode("Potential fraud").tolist()

neighbors = fg.find_neighbors(embedding=query_embedding, col="embedding", k=5)
```

Similarity search: embed the query with the same model, get the k nearest documents.

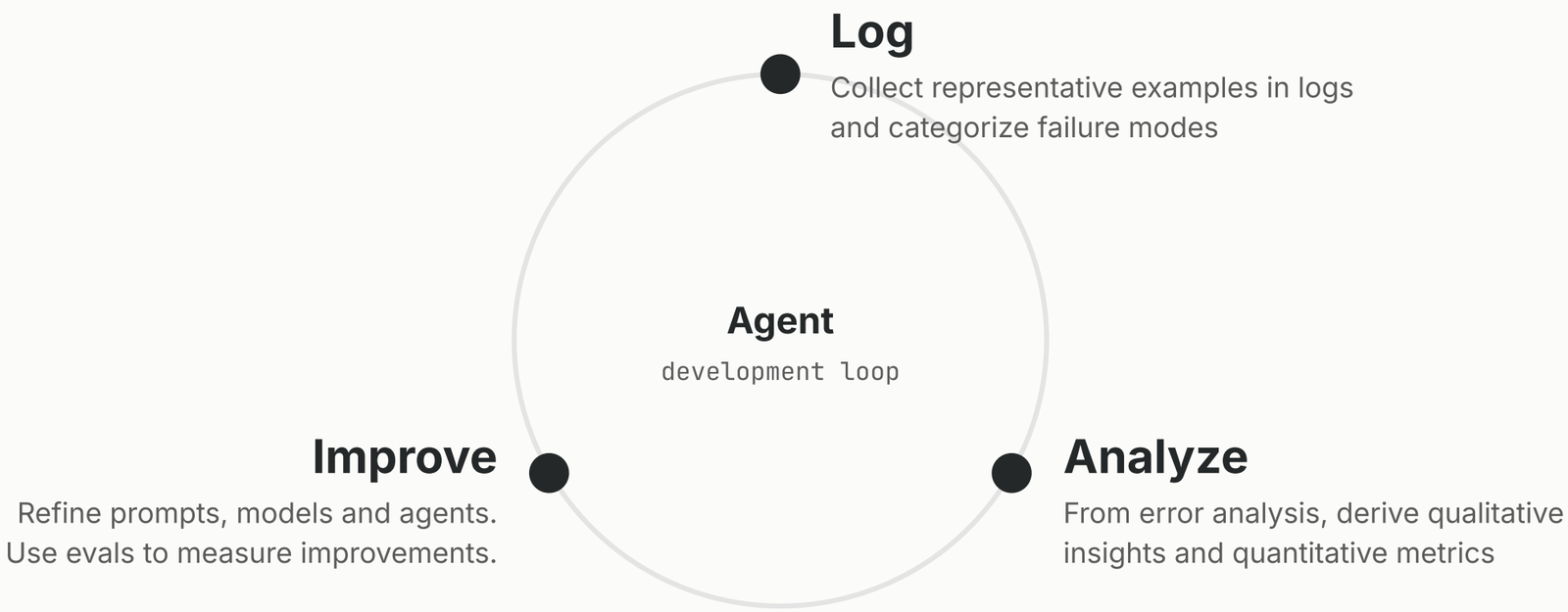
// LLM agents as online inference pipelines



Pipelines keep it fresh: clicks and actions, the application DB, docs and files. The agent's traces are logged too.

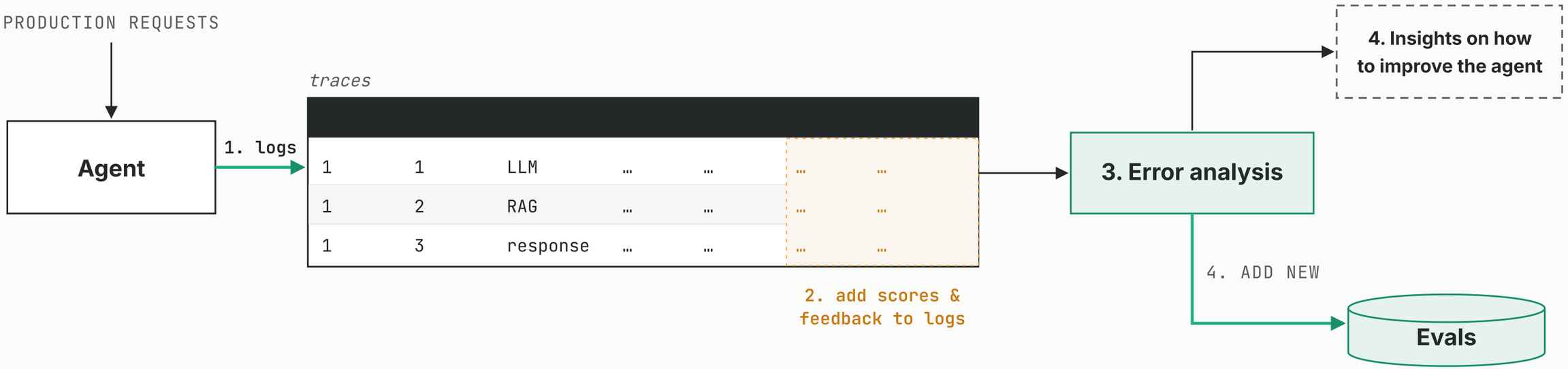
HW

// Software development for agents: error analysis and evals



Improve an agent like any ML system: from its logs, with measurements.

// Error analysis for LLM agents



4. ...gives insights on how to improve the agent, and new evals.

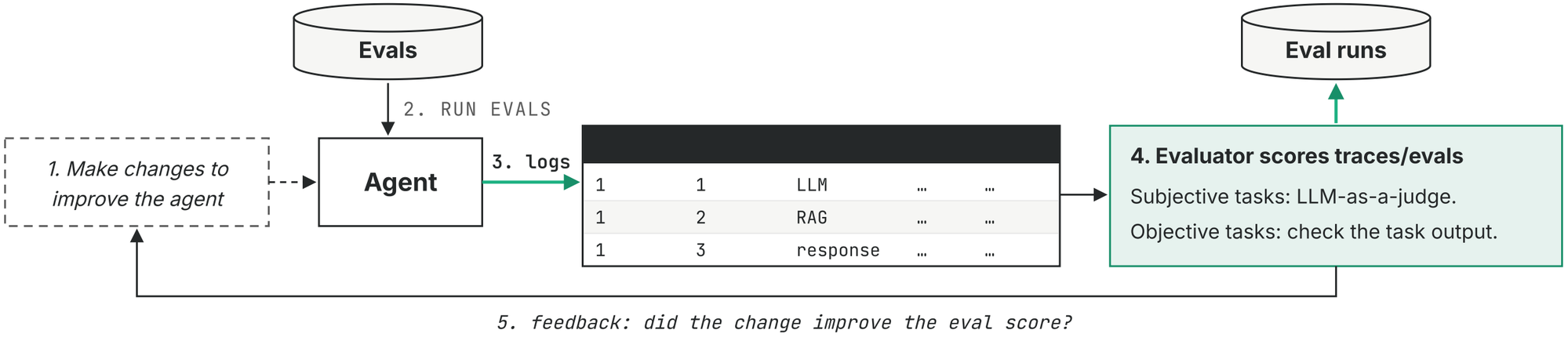
// Evals to incrementally improve LLM agents

evals

eval_id	event_ts	task	prompt	expected_response
1	2025-03-17 10:00	feature pipeline	Write a feature pipeline for the Titanic Dataset	Created feature group with 891 rows.
…	…	…	…	…

eval_runs

eval_id	event_ts	response	feedback	score
1	2025-03-18 10:00	Created feature group with 891 rows.	feature group creation worked.	5
…	…	…	…	…



5. Feedback: did the change improve the eval score, or not?

HW

05

Taxonomy of data transformations

FEATURE → TRAINING → INFERENCE

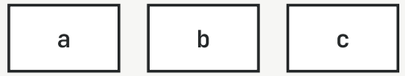
HW

DATA TRANSFORMATIONS

Useful Properties of Data Transformations for ML

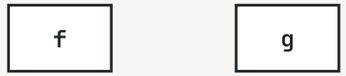
Modular

Small named units to look up, audit and swap.



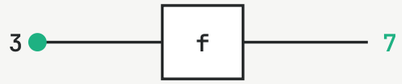
Composition

New transformations built from existing ones.



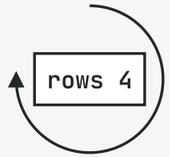
Determinism

Same inputs, params and state, same outputs.



Idempotent

Retry without side effects, stateful or not.



Scalable

Pandas UDFs / PySpark scales training data



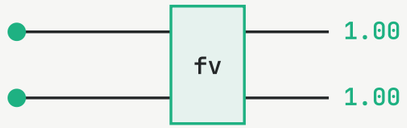
Fit on training data

Encoders and scalers take training statistics.



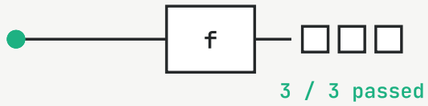
No skew

Same SQL and same UDFs run offline and online

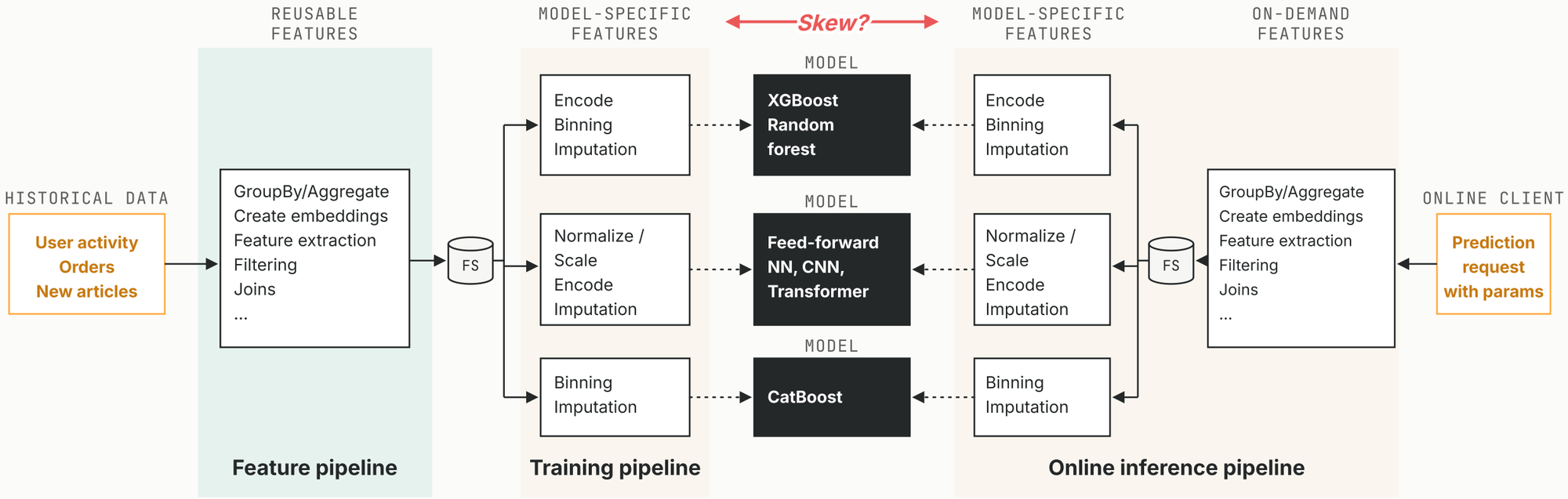


Testable

Unit and integration tests for data transformations.

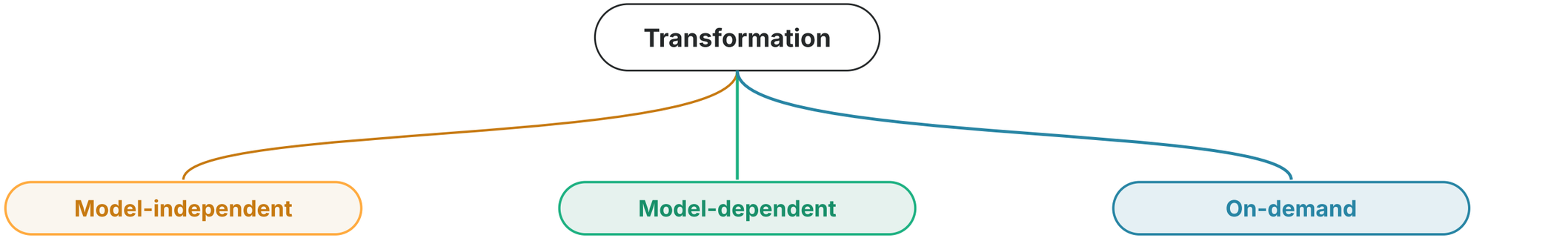


// Where do data transformations go when I have a feature store?



On-demand features are computed online, from the prediction request's parameters.

// A taxonomy of data transformations for AI



Data engineering

Aggregations, windowed counts, RFM.

PRODUCES Reusable features

TECHNOLOGIES Spark, Flink, Pandas, DBT/SQL, Polars, DuckDB, Feldera

Fit on training data

Feature encoding, scaling, imputation. Parameterized by the training data.

PRODUCES Model-specific features

TECHNOLOGIES Python (Pandas, Scikit-Learn)

Need request-time data

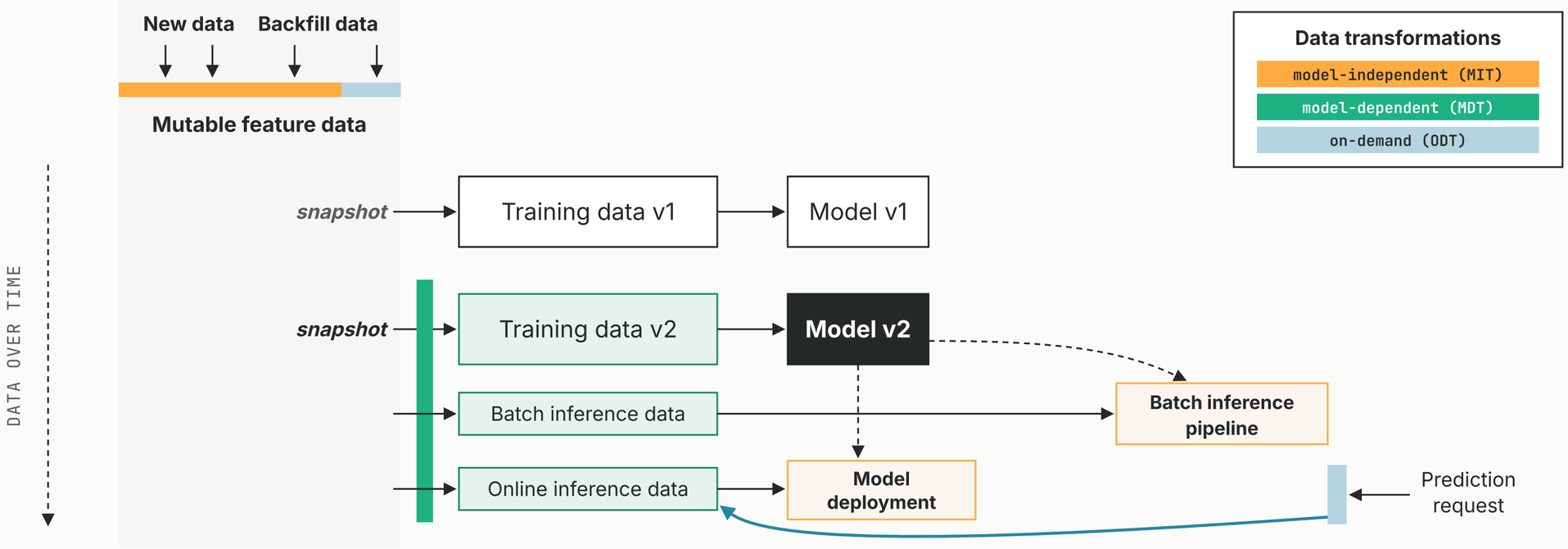
Computed online, at request time.

PRODUCES On-demand features

TECHNOLOGIES SQL, Python, custom

Reference: hopsworks.ai/post/a-taxonomy-for-data-transformations-in-ai-systems

// Data transformations and the feature store



Online, on-demand transformations run on the prediction request, then model-dependent ones, before the model deployment.

HW

06

MLOps principles

FEATURE → TRAINING → INFERENCE

// MLOps principles

AI-enabled products evolve over time

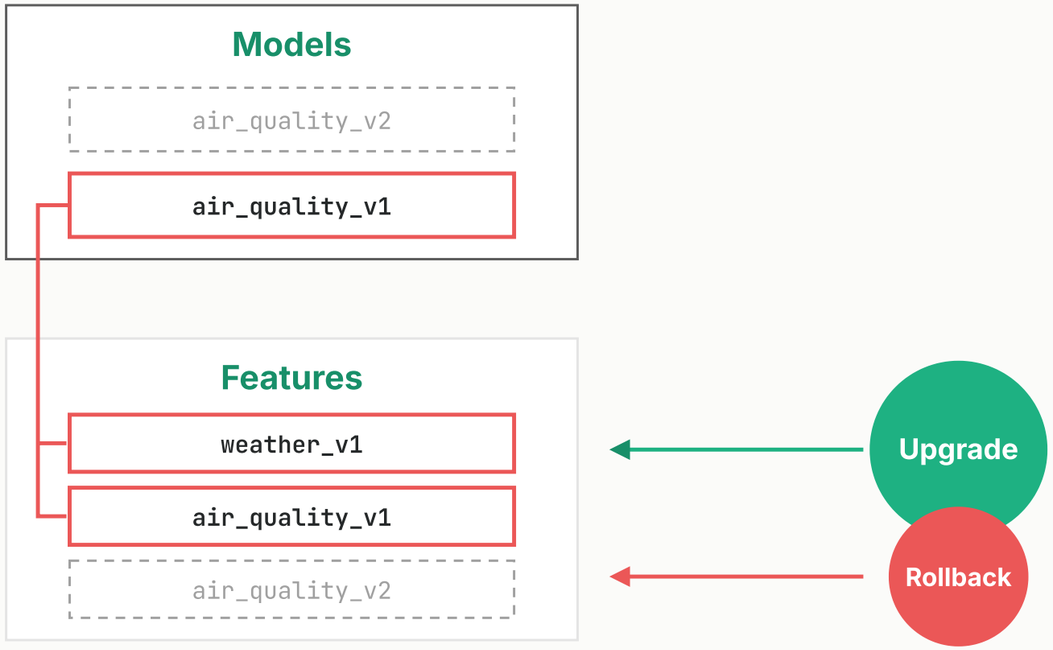
- The available input data (features) change over time
- The target you are trying to predict changes over time

Automate the testing and deployment of ML-enabled products

- **Features and models must be tested and versioned**
- Tests should run automatically, as part of a CI/CD workflow
- Features and models should be monitored in production

HW

// Feature and model versions are tightly coupled



ROLLED BACK TO

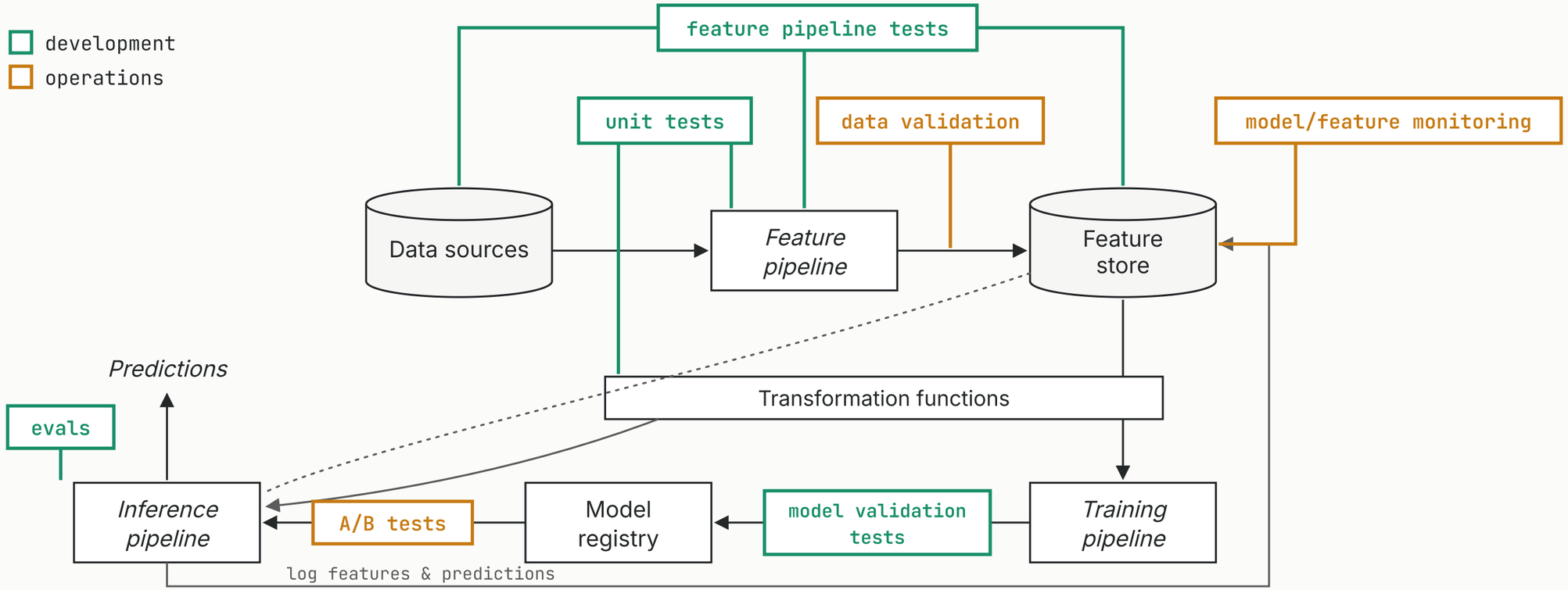
`air_quality_v1`

reads weather_v1, air_quality_v1

Rollback: back to air_quality_v1, which must read air_quality_v1 features again. Versioning models alone is not enough.

HW

// Testing pipelines with a feature store



Operations: data validation on writes, feature and model monitoring, A/B tests.

HW

// 12 fallacies of MLOps

01 Data is static

02 Do it all in one ML pipeline

03 You need an ML orchestrator for ML pipelines

04 All data transformations for AI are created equal

05 There is no need for a feature store

06 Experiment tracking is needed for MLOps

07 MLOps is just DevOps for ML

08 Versioning models is enough for safe upgrade/rollback

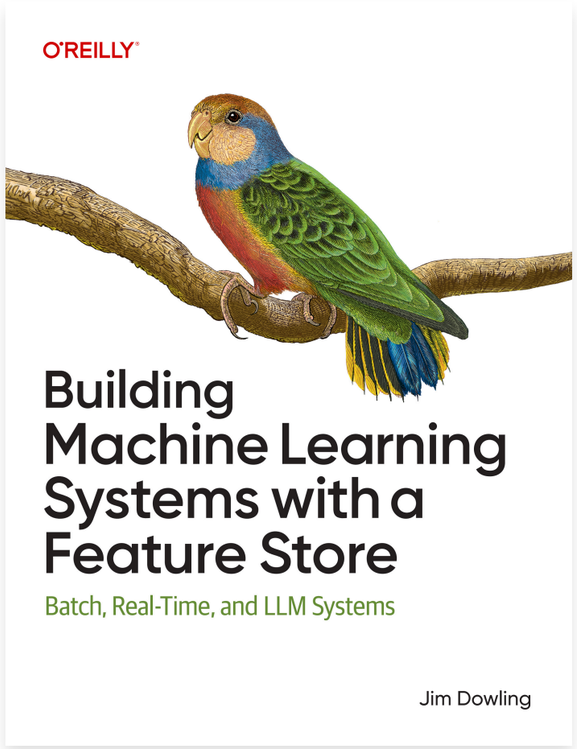
09 There is no need for data versioning

10 The model signature is the API for model deployments

11 Prediction latency is the time taken for the model prediction

12 LLMOps is not MLOps

// References



THE BOOK

Building Machine Learning Systems with a Feature Store

Batch, Real-Time, and LLM Systems · Jim Dowling · O'Reilly

FURTHER READING

The Hopsworks Feature Store for Machine Learning

SIGMOD 2024 · dl.acm.org/doi/10.1145/3626246.3653389

these slides
slides.hops.io/2026-11/id2223

HOPSWORKS



Join our Slack community

Explore our latest tutorials

Ask us any questions

public-hopsworks.slack.com